

Rising Waters: A Machine Learning Approach to Flood Prediction

Abstract

Floods are one of the most devastating natural disasters, causing significant loss of life, property damage, and economic disruption across the world. Early prediction of flood events plays a crucial role in disaster preparedness and mitigation by enabling authorities to take timely preventive measures. However, traditional flood forecasting methods often face challenges in accurately predicting flood occurrences due to the complex relationship between weather conditions and environmental factors.

The Rising Waters: A Machine Learning Approach to Flood Prediction project is developed to provide an intelligent flood prediction system using Machine Learning techniques. The system utilizes historical weather data containing meteorological parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, humidity, and other environmental factors to predict the likelihood of flood occurrence. By analysing these parameters, the system assists disaster management authorities and decision-makers in identifying flood-prone situations at an early stage.

The project follows a structured Machine Learning workflow consisting of data collection, exploratory data analysis, data preprocessing, feature engineering, model training, evaluation, and deployment. Multiple classification algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost were developed and compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. Among these models, the Decision Tree classifier achieved an accuracy of 95.65% and was selected as the final model for deployment.

The trained model was integrated into a Flask-based web application that allows users to enter weather-related parameters and instantly receive flood risk predictions. The application displays the predicted flood risk level, confidence score, evacuation advisory, and safety recommendations through a simple and responsive user interface. The system also incorporates data visualization techniques to better understand feature distributions, correlations, and model performance.

This project demonstrates the practical application of Machine Learning in disaster management and early warning systems. It provides a reliable and efficient solution for flood risk prediction and has the potential to be enhanced further through cloud deployment, real-time weather API integration, GIS-based flood mapping, IoT-enabled environmental monitoring, and mobile application support for large-scale disaster management.

Project Description:

Floods are among the most devastating natural disasters, claiming thousands of lives and causing extensive damage to infrastructure, agriculture, and the environment every year. Despite advancements in weather forecasting, accurately predicting flood events remains a significant challenge due to the complex relationship between rainfall patterns, meteorological conditions, and environmental factors. Delayed or inaccurate predictions often result in insufficient preparation time, leading to increased economic losses and threats to public safety.

Rising Waters: A Machine Learning Approach to Flood Prediction addresses this challenge by developing an intelligent flood prediction system using Machine Learning techniques. The system is trained on historical weather data containing parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, humidity, and other meteorological features. Multiple classification algorithms, including **Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost**, are trained and evaluated to identify the most accurate model for predicting flood risk.

The best-performing model is integrated into a **Flask-based web application**, enabling users to enter weather-related parameters and instantly receive flood risk predictions. The application provides prediction confidence, flood risk classification, evacuation advisory, and safety recommendations through a responsive and user-friendly interface. The project is designed to support future deployment on cloud platforms such as **IBM Cloud**, making it scalable, accessible, and suitable for real-world disaster management applications.

Scenario 1: Early Flood Warning and Evacuation Planning

A meteorologist collects current weather information, including rainfall, cloud visibility, temperature, and humidity, for a flood-prone district. After entering these parameters into the application, the Machine Learning model predicts a **high probability of flooding**. Based on the prediction, disaster management authorities issue early evacuation advisories, activate emergency response teams, and alert nearby communities to reduce the potential impact of the flood.

Scenario 2: Disaster Response and Resource Allocation

During the monsoon season, disaster management officials continuously monitor weather conditions across multiple regions. By entering the latest meteorological data into the web application, they receive instant flood risk predictions for each location. The system helps authorities prioritize emergency resources, deploy rescue teams efficiently, and allocate relief materials to high-risk areas before flooding occurs.

Scenario 3: Model Validation and Performance Assessment

A government disaster management analyst evaluates the developed Machine Learning models using historical flood event data. Multiple classification algorithms—Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost—are compared using evaluation metrics such as Accuracy, Precision, Recall, F1-Score, and ROC-AUC. The **Decision Tree classifier achieved an accuracy of 95.65%**, making it the best-performing model for deployment. The analyst confirms that the system provides reliable flood predictions and can effectively support disaster preparedness and early warning initiatives.

Technical Architecture

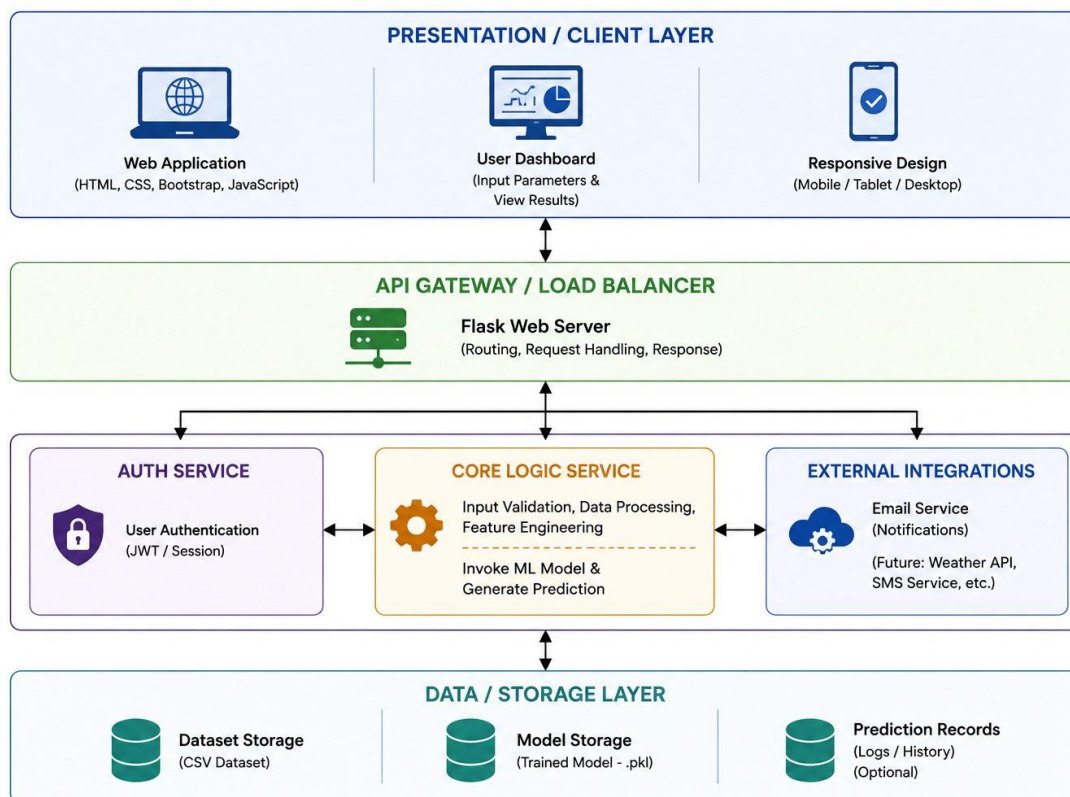


Figure 1: Technical Architecture of the Rising Waters System

Description

The Rising Waters system follows a multi-layer architecture consisting of the Presentation Layer, API Gateway Layer, Core Logic Layer, Integration Layer, and Data Storage Layer. Each layer performs a specific function to provide accurate flood risk predictions based on weather and environmental parameters. The architecture ensures efficient communication between the user interface, Flask backend, Machine Learning model, and data storage, resulting in a reliable and scalable flood prediction system.

1. Presentation Layer (Frontend)

The Presentation Layer is responsible for user interaction and provides an intuitive, responsive web interface developed using HTML, CSS, Bootstrap, and JavaScript. Users can enter weather-related parameters such as annual rainfall, seasonal rainfall, cloud visibility, temperature, humidity, and other meteorological features required for flood prediction.

The Presentation Layer also provides:

- User-friendly web interface
- Responsive design for Desktop, Tablet, and Mobile devices
- Weather parameter input forms
- Prediction result visualization
- Easy navigation throughout the application

2. API Gateway Layer (Flask Backend)

The API Gateway Layer is implemented using the Flask Web Framework. It acts as the communication bridge between the frontend and the Machine Learning components by handling incoming requests, routing them to the appropriate services, and returning prediction results.

backend is responsible for:

- Receiving user input from the web interface
- Request routing and response handling
- Server-side input validation
- Invoking the prediction pipeline
- Returning prediction results to the user interface
- Handling application errors and exceptions

3. Core Logic Layer

The Core Logic Layer performs all preprocessing and Machine Learning operations required for flood prediction. It validates user inputs, preprocesses weather parameters, applies feature scaling using the trained StandardScaler, and loads the trained Decision Tree model to generate predictions.

The workflow of this layer includes:

- Input validation and data preprocessing
- Feature engineering and scaling
- Loading the trained Decision Tree model
- Predicting flood risk
- Calculating prediction confidence
- Generating flood risk classification
- Preparing safety recommendations and evacuation advisory

The deployed Decision Tree classifier achieved an overall prediction accuracy of 95.65%, making it a reliable model for flood risk prediction.

4. External Integration Layer

The External Integration Layer supports future enhancements that extend the capabilities of the flood prediction system. Although these services are not currently implemented, the architecture is designed to integrate seamlessly with external platforms and services.

Future integrations include:

- Weather API for real-time meteorological data
- IoT sensors for rainfall and river water-level monitoring
- SMS notification service for emergency alerts
- Email notification service for disaster warnings
- IBM Cloud deployment for remote accessibility
- GIS mapping services for flood visualization

These integrations will improve the accuracy, scalability, and real-time capabilities of the system.

5. Data Storage Layer

The Data Storage Layer stores all datasets, trained Machine Learning models, and prediction-related information required by the application.

This layer includes:

- Historical Weather Dataset (flood_dataset.xlsx)
- Trained Decision Tree Model (flood_prediction.pkl)
- StandardScaler (scaler.pkl)
- Model Evaluation Results (model_comparison.csv)

- Prediction records and logs (future enhancement)

The trained model is stored using Pickle, enabling fast loading and prediction without retraining the model each time the application starts.

6. User Output

Finally, the application displays comprehensive flood prediction results through an interactive and user-friendly interface.

The output includes:

- Flood Risk Prediction (High Risk / Low Risk)
- Prediction Confidence Score
- Flood Risk Percentage
- Flood Status Indicator
- Evacuation Advisory
- Safety Recommendations
- Weather-based Risk Assessment

This provides users and disaster management authorities with meaningful insights rather than displaying only the predicted flood status, enabling informed decision-making and timely disaster preparedness.

Project Workflow

The Rising Waters system follows a structured workflow that transforms user-provided weather and meteorological parameters into accurate flood risk predictions using Machine Learning. The workflow consists of user input collection, input validation, data preprocessing, feature scaling, machine learning prediction, confidence calculation, recommendation generation, and result visualization. Each stage is designed to ensure reliable, accurate, and efficient flood risk assessment, enabling users and disaster management authorities to make timely decisions for flood preparedness and emergency response

The overall workflow of the system is illustrated below.

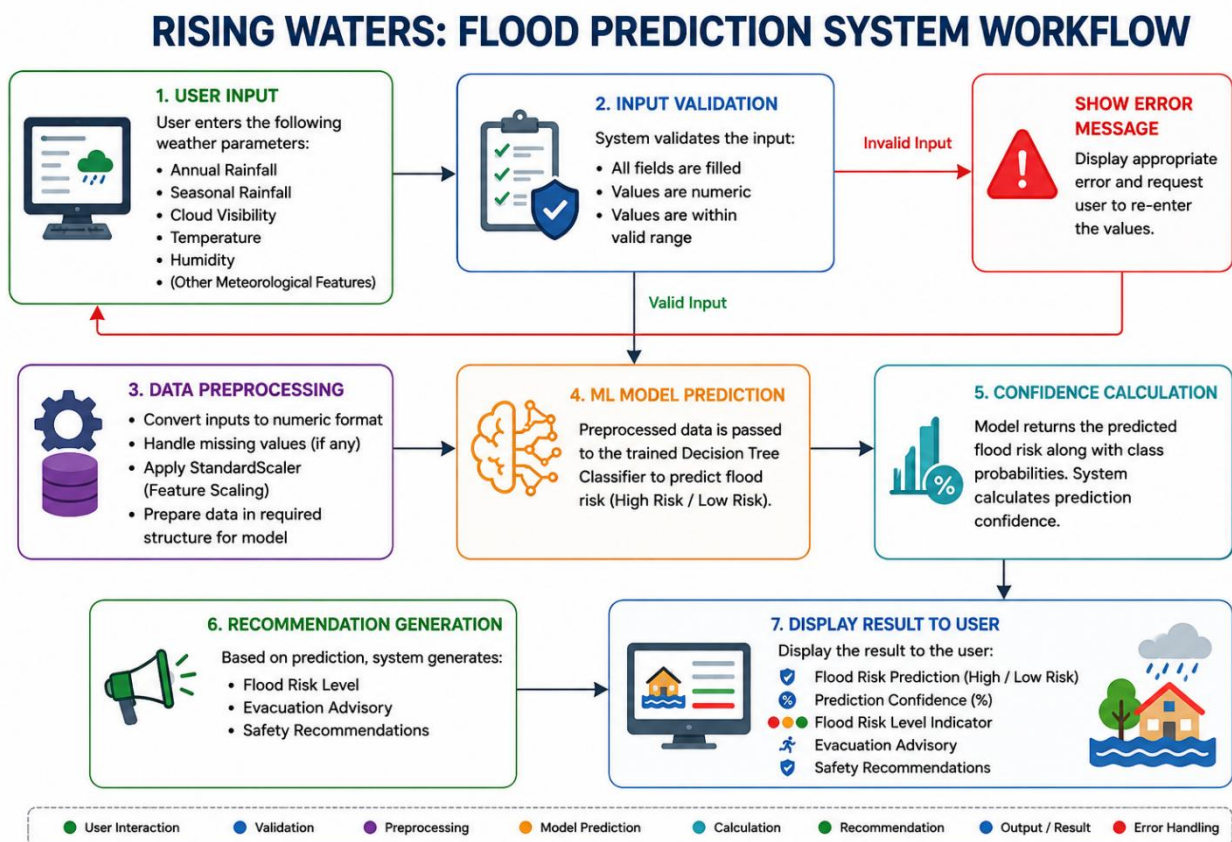


Figure 2: Workflow of the Rising Waters System

Step 1: User Input

The workflow begins when the user enters the required weather and rainfall parameters through the **Flood Risk Prediction Panel**. The application accepts the following inputs:

- Temperature (°C)
- Humidity (%)
- Cloud Cover
- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June–September Rainfall
- October–December Rainfall
- Average Rainfall in June
- Sub-divisional Rainfall

These parameters are collected from historical meteorological observations and are used as input features for the trained Machine Learning model. After entering all the required values, the user clicks the **"Predict Flood Risk"** button to initiate the prediction process.

Step 2: Input Validation

After the user submits the weather parameters, the application performs input validation to ensure the data is complete, accurate, and suitable for prediction. The Flask backend verifies that all required fields have been filled, checks that the entered values are numeric, and confirms that they fall within the valid ranges defined by the training dataset. This validation helps prevent incorrect or incomplete data from affecting the prediction process.

If invalid or missing values are detected, the application displays an appropriate error message and prompts the user to correct the inputs before proceeding. Only valid data is forwarded to the preprocessing stage for flood risk prediction.

The system validates that:

- All required input fields are completed.
- Values are numeric and in the correct format.
- Each parameter falls within the acceptable range.
- No missing or invalid values are submitted.

This validation ensures reliable predictions and improves the overall accuracy and stability of the flood prediction system.

Step 3: Data Preprocessing

After successful input validation, the system preprocesses the weather data to make it suitable for the trained Machine Learning model. The entered values are converted into the required numerical format, cleaned if necessary, and transformed using the saved StandardScaler to maintain consistency with the data used during model training. This preprocessing step ensures that all input features are standardized before prediction, thereby improving the accuracy and reliability of the model.

The preprocessing stage includes:

- Converting user inputs into the required numerical format.
- Applying the trained StandardScaler for feature scaling.
- Organizing the input data according to the model's expected feature structure.

- Preparing the processed data for the Decision Tree prediction model.

This standardized preprocessing pipeline ensures that the prediction process is consistent with the training workflow and helps generate accurate flood risk predictions.

Step 4: Machine Learning Prediction

The preprocessed data is forwarded to the trained Decision Tree Classification Model, which was selected as the best-performing algorithm after evaluating multiple Machine Learning models. The model analyzes the weather and rainfall parameters to determine the likelihood of flood occurrence and classifies the input as either High Flood Risk or Low Flood Risk.

The workflow of this layer includes:

- Receiving the preprocessed and scaled input features.
- Loading the trained Decision Tree model.
- Analyzing weather and rainfall conditions.
- Predicting the flood risk category.
- Returning the prediction result to the Flask application.

The trained Decision Tree model achieved an overall prediction accuracy of 95.65%, making it a reliable solution for flood risk prediction.

5. Prediction Confidence Calculation

After generating the flood risk prediction, the system calculates the confidence score associated with the predicted class. The confidence score indicates the probability of the prediction and helps users understand the reliability of the model's output. A higher confidence value reflects greater certainty in the prediction.

The confidence calculation module:

- Computes prediction probability.
- Determines the confidence percentage.
- Measures the reliability of the prediction.

- Sends the confidence score to the result display module.

6. Recommendation Generation

Based on the predicted flood risk, the application generates suitable recommendations and safety guidelines for the user. If the predicted risk is high, the system provides evacuation advice and precautionary measures. If the predicted risk is low, it recommends continued monitoring of weather conditions and maintaining general preparedness.

This module provides:

- Flood Risk Status
- Evacuation Advisory
- Safety Recommendations
- Disaster Preparedness Guidelines
- Weather-based Risk Assessment

These recommendations help users and disaster management authorities take appropriate actions based on the predicted flood conditions.

7. Display Result to User

Finally, the application presents the prediction results through a responsive and user-friendly web interface. The results are displayed in an easy-to-understand format, enabling users to quickly assess the flood situation and make informed decisions.

The application displays:

- Flood Risk Prediction (High Risk / Low Risk)
- Prediction Confidence Score
- Flood Risk Percentage
- Flood Status Indicator
- Evacuation Advisory
- Safety Recommendations

This provides users with a comprehensive flood risk assessment instead of displaying only the predicted flood status, making the system more informative and practical for disaster preparedness and emergency response.

Milestone 1: Project Setup and Dataset Preparation

Objective

The objective of this milestone was to establish the development environment required for building the Flood Prediction System, collect a suitable flood prediction dataset, and perform an initial exploration of the dataset before proceeding with preprocessing and Machine Learning model development.

Activity 1.1: Development Environment Setup

Objective

To configure the software environment and install all required tools, libraries, and frameworks necessary for developing the Machine Learning-based flood prediction system.

Tools Used

Software / Tool	Purpose
Python 3.12	Programming Language
Jupyter Notebook	Data Analysis and Model Development
Visual Studio Code	Flask Application Development
Flask	Web Application Framework
Pandas	Data Manipulation
NumPy	Numerical Computing
Matplotlib	Data Visualization
Scikit-learn	Machine Learning Algorithms

XGBoost	Boosting Classification Algorithm
Pickle	Model Serialization
Git & GitHub	Version Control

Activities Performed

- Installed Python and configured the development environment.
- Installed all required Python libraries.
- Created the project folder structure.
- Configured Jupyter Notebook for data analysis.
- Configured Visual Studio Code for Flask development.
- Installed Flask and Machine Learning libraries.
- Verified successful installation of all required packages.

Activity 1.2: Dataset Collection

Objective

To obtain a reliable dataset for training the flood prediction model.

Property	Value
Dataset Name	Flood Prediction Dataset
Source	Kaggle
Records	Historical flood and weather observations
Features	Weather and rainfall parameters

Property	Value
Target Classes	Flood (Yes / No)

Dataset Information

Input Features

- Temperature
- Humidity
- Cloud Cover
- Annual Rainfall
- January–February Rainfall
- March–May Rainfall
- June–September Rainfall
- October–December Rainfall
- Average Rainfall in June
- Sub-divisional Rainfall

Output

- Flood Risk (Flood / No Flood)

Activity 1.3: Data Exploration and Analysis

Objective

To understand the characteristics of the dataset before preprocessing and Machine Learning model training.

Activities Performed

- Loaded the dataset using the Pandas library.

- Explored the dataset structure and dimensions.
- Verified data types of all features.
- Checked for missing values.
- Examined duplicate records.
- Performed descriptive statistical analysis.
- Analyzed feature distributions using visualization techniques.
- Identified relationships among weather parameters.
- Verified the target variable distribution.

Dataset Summary

Parameter	Value
Total Samples	115
Total Input Features	10
Target Variable	Flood
Total Columns	11
Flood Cases (1)	16
No Flood Cases (0)	99
Data Types	Integer and Float
Missing Values	0
Duplicate Records	0

Dataset Shape: (115, 11)

Column Names:

['Temp', 'Humidity', 'Cloud Cover', 'ANNUAL', 'Jan-Feb', 'Mar-May', 'Jun-Sep', 'Oct-Dec', 'avgjune', 'sub', 'flood']

Data Types:

```
Temp          int64
Humidity      int64
Cloud Cover   int64
ANNUAL        float64
Jan-Feb       float64
Mar-May       float64
Jun-Sep       float64
Oct-Dec       float64
avgjune       float64
sub           float64
flood         int64
dtype: object
```

Target Column ('flood') Counts:

```
flood
0    99
1    16
```

Name: count, dtype: int64

First 5 Rows:

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

Milestone 2: Data Analysis, Preprocessing, and Machine Learning Model Development

Objective

The objective of this milestone was to analyze the flood prediction dataset, preprocess the data to improve its quality, develop multiple Machine Learning classification models, evaluate their performance, and select the best-performing model for deployment in the web application.

Activity 2.1: Data Visualization and Analysis

Objective

To understand the characteristics of the flood prediction dataset by performing exploratory data analysis and identifying relationships among weather and rainfall parameters.

Activities Performed

- Imported the required Python libraries.
- Loaded the flood prediction dataset.
- Explored dataset dimensions and feature information.
- Performed descriptive statistical analysis.
- Visualized feature distributions using histograms.
- Generated correlation heatmaps.
- Created scatter plots and box plots to identify patterns.
- Analyzed the distribution of the target variable (Flood).

Visualizations Generated

- Dataset Overview
 - Histogram of Numerical Features
 - Correlation Heatmap
 - Feature Distribution Graphs
 - Box Plot for Outlier Detection
 - Flood Class Distribution
-

Activity 2.2: Data Preprocessing

Objective

To prepare the dataset for Machine Learning model training by cleaning, transforming, and scaling the data.

Preprocessing Activities

- Checked for missing values.
- Verified duplicate records.
- Identified and handled outliers.
- Converted all input features into numerical format.
- Performed feature scaling using StandardScaler.
- Split the dataset into training and testing datasets.

Preprocessing Techniques

Technique	Purpose
Missing Value Analysis	Improve data quality
Duplicate Record Check	Remove redundant data
Outlier Detection	Improve model performance
Feature Scaling	Normalize numerical features
Train-Test Split	Model training and evaluation

Activity 2.3: Machine Learning Model Development

Objective

To train and compare multiple Machine Learning classification algorithms for flood prediction.

Models Developed

- Decision Tree Classifier
- Random Forest Classifier
- K-Nearest Neighbors (KNN)
- XGBoost Classifier

Each model was trained using the preprocessed dataset and evaluated using standard classification metrics.

Activity 2.4: Model Evaluation

Objective

To compare the performance of all trained Machine Learning models and identify the most accurate model for deployment.

Evaluation Metrics

The models were evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
- ROC-AUC Score

Model Performance

Machine Learning Model	Accuracy	Precision	Recall	F1 Score	ROC-AUC
Decision Tree	95.65%	100%	66.67%	80.00%	83.33%
Random Forest	95.65%	100%	66.67%	80.00%	94.17%
K-Nearest Neighbors	86.96%	50.00%	33.33%	40.00%	78.33%
XGBoost	86.96%	0.00%	0.00%	0.00%	50.00%

Activity 2.5: Model Selection

Objective

To select the most suitable Machine Learning model for deployment in the flood prediction web application.

Although both Decision Tree and Random Forest achieved the highest prediction accuracy of 95.65%, the Decision Tree Classifier was selected for deployment because of its excellent performance, lower computational complexity, faster prediction time, and simpler model structure. The trained model was serialized using Pickle and integrated into the Flask web application for real-time flood prediction.

Activity 2.6: Model Serialization

Objective

To save the trained Machine Learning model and preprocessing objects for deployment.

Files Generated

File	Purpose
flood_prediction.pkl	Trained Decision Tree model
scaler.pkl	StandardScaler used during preprocessing
model_comparison.csv	Performance comparison of all trained models

The saved model and scaler are loaded automatically by the Flask application during runtime, enabling real-time flood prediction without retraining the model.

Milestone 3: Web Application Development and System Integration

Objective

The objective of this milestone was to develop a web-based flood prediction application by integrating the trained Machine Learning model with a Flask backend and a responsive frontend. The application enables users to enter weather parameters, performs real-time flood risk prediction, and displays prediction confidence along with evacuation advisories and safety recommendations.

Activity 3.1: User Interface Development

Objective

To develop a responsive and user-friendly web interface for entering weather parameters and displaying flood prediction results.

Frontend Technologies Used

- HTML5
- CSS3
- Bootstrap 5
- JavaScript

Developed Pages

Web Page	Purpose
Home Page	Introduction to the Rising Waters project
Prediction Panel	Accepts weather and rainfall parameters from users
Result Page	Displays flood prediction, confidence score, and recommendations
About Project	Provides project overview, algorithms, and workflow
Error Page	Handles invalid routes and application errors

Activity 3.2: Flask Backend Development

Objective

To implement the backend logic responsible for processing user requests and integrating the trained Machine Learning model.

Backend Responsibilities

- Receive weather parameters from the frontend.
- Validate user inputs.

- Perform data preprocessing and feature scaling.
 - Load the trained Decision Tree model.
 - Generate flood risk predictions.
 - Calculate prediction confidence.
 - Return prediction results to the frontend.
 - Handle invalid requests and application errors.
-

Activity 3.3: Machine Learning Model Integration

Objective

To integrate the trained Decision Tree model into the Flask application for real-time flood prediction.

Integration Process

- Load the trained **Decision Tree** model (flood_prediction.pkl).
- Load the saved **StandardScaler** (scaler.pkl).
- Preprocess user input using the same pipeline as the training phase.

- Pass the processed data to the model.
- Generate flood risk prediction.
- Return prediction confidence and risk classification.

The deployed Decision Tree model achieved an overall prediction accuracy of **95.65%** and provides reliable flood risk predictions.

Activity 3.4: Input Validation

Objective

To ensure that only valid weather parameters are processed by the system.

Validation Techniques

Client-Side Validation

Implemented using JavaScript.

Validation checks include:

- Mandatory input fields
- Numeric values only

- Acceptable parameter ranges
- User-friendly validation messages

Server-Side Validation

Implemented using Flask.

Validation checks include:

- Data type verification
 - Range validation
 - Missing parameter handling
 - Exception and error management
-

Activity 3.5: Prediction Result Generation

Objective

To present meaningful flood prediction results to the user.

Prediction Output

After processing the input values, the application displays:

- Flood Risk Prediction (High Risk / Low Risk)
- Flood Risk Percentage
- Prediction Confidence (%)
- Flood Status Indicator
- Evacuation Advisory
- Safety Recommendations

The confidence score is calculated using the probability values generated by the trained Decision Tree classifier.

Activity 3.6: Recommendation Module

Objective

To provide users with meaningful guidance based on the predicted flood risk.

Information Displayed

Information	Description
Flood Risk	High Risk / Low Risk
Confidence Score	Probability of the prediction
Flood Status	Current flood condition
Evacuation Advisory	Whether evacuation is recommended
Safety Recommendations	Preventive actions based on prediction

The recommendation module generates customized guidance based on the predicted flood risk, helping users and disaster management authorities respond appropriately.

Activity 3.7: System Integration

Objective

To integrate the frontend, backend, preprocessing pipeline, and Machine Learning model into a complete flood prediction application.

System Workflow

User Opens Application



Enter Weather Parameters



Input Validation



Flask Backend Receives Request



Data Preprocessing



Feature Scaling



Decision Tree Prediction



Confidence Calculation



Recommendation Generation



Display Flood Prediction Results

Milestone 4: Testing and Performance Evaluation

Objective

The objective of this milestone was to evaluate the functionality, reliability, accuracy, and performance of the Rising Waters application. Various testing techniques were performed to verify that each module of the system operated correctly and that the Machine Learning model produced accurate flood risk predictions.

Activity 4.1: Functional Testing

Objective

To verify that all major functionalities of the application operate as expected.

Functional Test Cases

Test Case	Description	Expected Result	Status
Home Page Loading	Verify application homepage loads successfully	Home page displayed correctly	Passed
User Input	Verify users can enter weather parameters	Input accepted successfully	Passed
Prediction Request	Verify prediction request reaches backend	Backend processes request successfully	Passed
Flood Risk Prediction	Verify flood prediction is displayed	Correct flood risk prediction shown	Passed
Prediction Confidence	Verify confidence score is displayed	Confidence percentage shown correctly	Passed
Result Display	Verify recommendations and advisory are displayed	Results displayed successfully	Passed
Reset Function	Verify reset button clears all input fields	Form resets successfully	Passed

Activity 4.2: Input Validation Testing

Objective

To ensure that invalid or incomplete user inputs are properly handled.

Validation Scenarios

Validation	Expected Behaviour	Status
Empty Fields	Display validation message	Passed
Negative Values	Reject invalid input	Passed
Out-of-Range Values	Display error message	Passed
Non-Numeric Input	Prevent submission	Passed
Missing Parameters	Server rejects invalid request	Passed

Activity 4.3: Machine Learning Model Evaluation

Objective

To evaluate the prediction performance of the trained Machine Learning models.

Models Compared

Machine Learning Model	Accuracy
Decision Tree	95.65%
Random Forest	95.65%
K-Nearest Neighbors (KNN)	86.96%
XGBoost	86.96%

The Decision Tree classifier achieved the highest overall performance for deployment based on accuracy, computational efficiency, and prediction speed. The trained model was selected as the final deployment model for the Flask application.

Activity 4.4: Performance Testing

Objective

To measure the application's response time, reliability, and stability.

Performance Results

Parameter	Result
Average Prediction Time	Less than 2 seconds
Model Loading	Successful
Response Generation	Immediate
Multiple Consecutive Predictions	Successfully Executed
Application Stability	Stable



[Home](#)
[Prediction Panel](#)
[About Project](#)


Flood Risk Prediction Panel

Temp

Range: 28.0 - 31.0

30.0

Humidity

Range: 70.0 - 79.0

74.0

Cloud Cover

Range: 30.0 - 44.0

36.0

ANNUAL

Range: 2068.8 - 4257.8

2937.5

Jan-Feb

Range: 0.3 - 98.3

20.5

Mar-May

Range: 89.9 - 915.2

342.0

Jun-Sep

Range: 1104.3 - 3451.3

1948.7

Oct-Dec

Range: 166.6 - 823.3

501.5

avgjune

Range: 65.6 - 366.1

211.03

sub

Range: 34.2 - 982.7

430.6

Reset Values

Predict Flood Risk


Parameters Guide

Temp / Humidity / Cloud Cover: Regional weather figures for the forecasting target day.

ANNUAL / sub: Aggregated annual and sub-divisional rainfall totals (in millimeters).

Jan-Feb / Mar-May / Jun-Sep / Oct-Dec: Seasonal rainfall splits representing climate cycles.

avgjune: Mean rainfall recorded in June, historically a key flooding indicator.


Session History

Activity 4.5: User Acceptance Testing (UAT)

Objective

To verify that the developed application satisfies the intended user requirements.

UAT Results

Requirement	Result
User-Friendly Interface	Passed
Accurate Flood Prediction	Passed
Input Validation	Passed
Prediction Confidence Display	Passed
Flood Risk Assessment	Passed
Safety Recommendations	Passed
Evacuation Advisory	Passed

Activity 4.6: Test Result Summary

Overall Testing Summary

Testing Type	Result
Functional Testing	Passed
Input Validation Testing	Passed
Machine Learning Evaluation	Passed
Performance Testing	Passed
User Acceptance Testing	Passed

Overall Model Accuracy

Decision Tree Classification Accuracy: 95.65%

Activity 4.7: Issues Identified and Resolved

During testing, a few implementation issues were identified and successfully resolved.

Issue	Resolution
Model integration with Flask	Successfully integrated using Pickle serialization.
Invalid user input handling	Added both client-side and server-side validation.
Prediction result formatting	Improved result display with confidence score, flood risk percentage, and recommendations.
Model loading efficiency	Loaded the model once during application startup to improve performance.
Feature scaling consistency	Used the saved StandardScaler during prediction to ensure consistent preprocessing.

Milestone 5: Project Demonstration and Conclusion

This milestone focuses on demonstrating the completed Rising Waters application, verifying that all proposed functionalities work as expected, and presenting the final outcomes of the project. The application was tested through multiple prediction scenarios to validate the performance of the Machine Learning model and the overall system workflow.

Activity 5.1: Application Deployment and Local Execution

Objective

To execute the **Rising Waters** application in the local development environment and verify that all application components function correctly.

Steps Performed

- Activated the Python virtual environment.
- Installed all required dependencies using **requirements.txt**.
- Started the Flask development server.
- Opened the application using the local host URL.
- Verified successful loading of the Home Page.
- Confirmed successful loading of the trained **Decision Tree** Machine Learning model and **StandardScaler**.
- Tested multiple flood prediction scenarios to verify system functionality.

Local Execution Command

```
python app.py
```

Local URL

```
http://127.0.0.1:5000
```

Activity 5.2: Demonstration of the Web Application

The Rising Waters application was demonstrated by entering weather and rainfall parameters through the web interface. The application processed the user inputs, performed data preprocessing and feature scaling, executed the trained Decision Tree Machine Learning model, and displayed the predicted flood risk along with prediction confidence, evacuation advisory, and safety recommendations.

Demonstrated Features

- Flood Risk Prediction
- Real-Time Machine Learning Prediction
- Prediction Confidence Display
- Flood Risk Percentage
- Evacuation Advisory
- Safety Recommendations
- Client-Side Validation
- Server-Side Validation
- Reset Functionality

Activity 5.3: Exploring the Website

Home Page

The Home Page is the main interface of the Rising Waters application. It introduces the project and provides easy navigation to the Prediction Panel and About Project pages. Developed using HTML5, CSS3, Bootstrap 5, and JavaScript, the interface is responsive, modern, and user-friendly. It also highlights the key features of the system and provides quick access to flood prediction.

Main Features

- Project overview and navigation menu.
- Open Prediction Panel for flood risk prediction.
- View Data Insights for dataset visualization.
- Dark/Light theme toggle.
- Highlights of key system features such as Machine Learning models, dynamic forms, and PDF report generation.

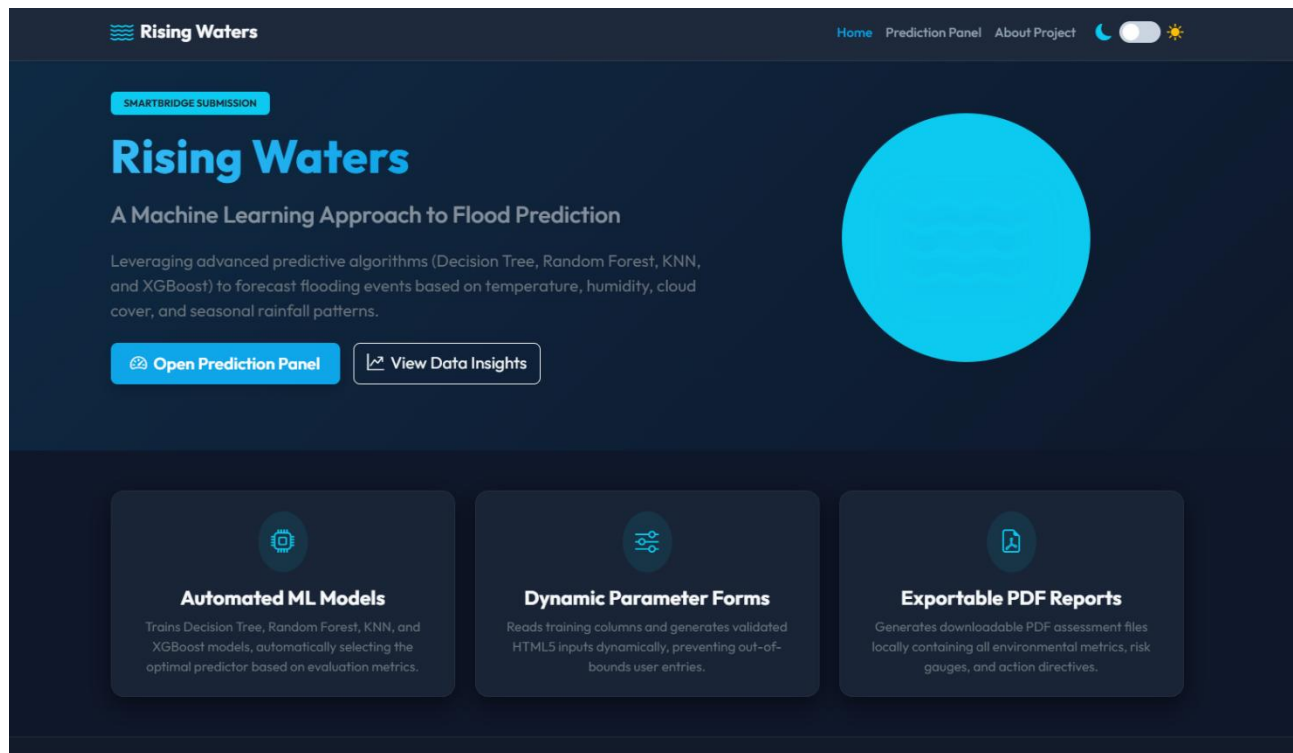


Figure 5.1: Rising Waters Home Page

Prediction Form

Description

The **Prediction Form** allows users to enter weather and rainfall parameters required for flood risk prediction. It includes input fields for **Temperature**, **Humidity**, **Cloud Cover**, **Annual Rainfall**, **Jan–Feb Rainfall**, **Mar–May Rainfall**, **Jun–Sep Rainfall**, **Oct–Dec Rainfall**, **Average June Rainfall**, and **Sub-divisional Rainfall**. The form also provides parameter guidance, session history, and input range validation to help users enter accurate values.

After entering valid inputs, users can click the **Predict Flood Risk** button to generate the flood risk assessment.


Rising Waters

[Home](#)
[Prediction Panel](#)
[About Project](#)





Flood Risk Prediction Panel

Temp

Range: 28.0 - 31.0

30.0

Humidity

Range: 70.0 - 79.0

74.0

Cloud Cover

Range: 30.0 - 44.0

36.0

ANNUAL

Range: 2068.8 - 4257.8

2937.5

Jan-Feb

Range: 0.3 - 98.1

20.5

Mar-May

Range: 89.9 - 915.2

342.0

Jun-Sep

Range: 1104.3 - 3451.3

1948.7

Oct-Dec

Range: 166.6 - 823.3

501.5

avgjune

Range: 65.6 - 366.1

211.03

sub

Range: 34.2 - 982.7

430.6

Reset Values

Predict Flood Risk


Parameters Guide

Temp / Humidity / Cloud Cover: Regional weather figures for the forecasting target day.

ANNUAL / sub: Aggregated annual and sub-divisional rainfall totals (in millimeters).

Jan-Feb / Mar-May / Jun-Sep / Oct-Dec: Seasonal rainfall splits representing climate cycles.

avgjune: Mean rainfall recorded in June, historically a key flooding indicator.


Session History

Figure 5.2: Flood Risk Prediction Form

Prediction Result Page

Description

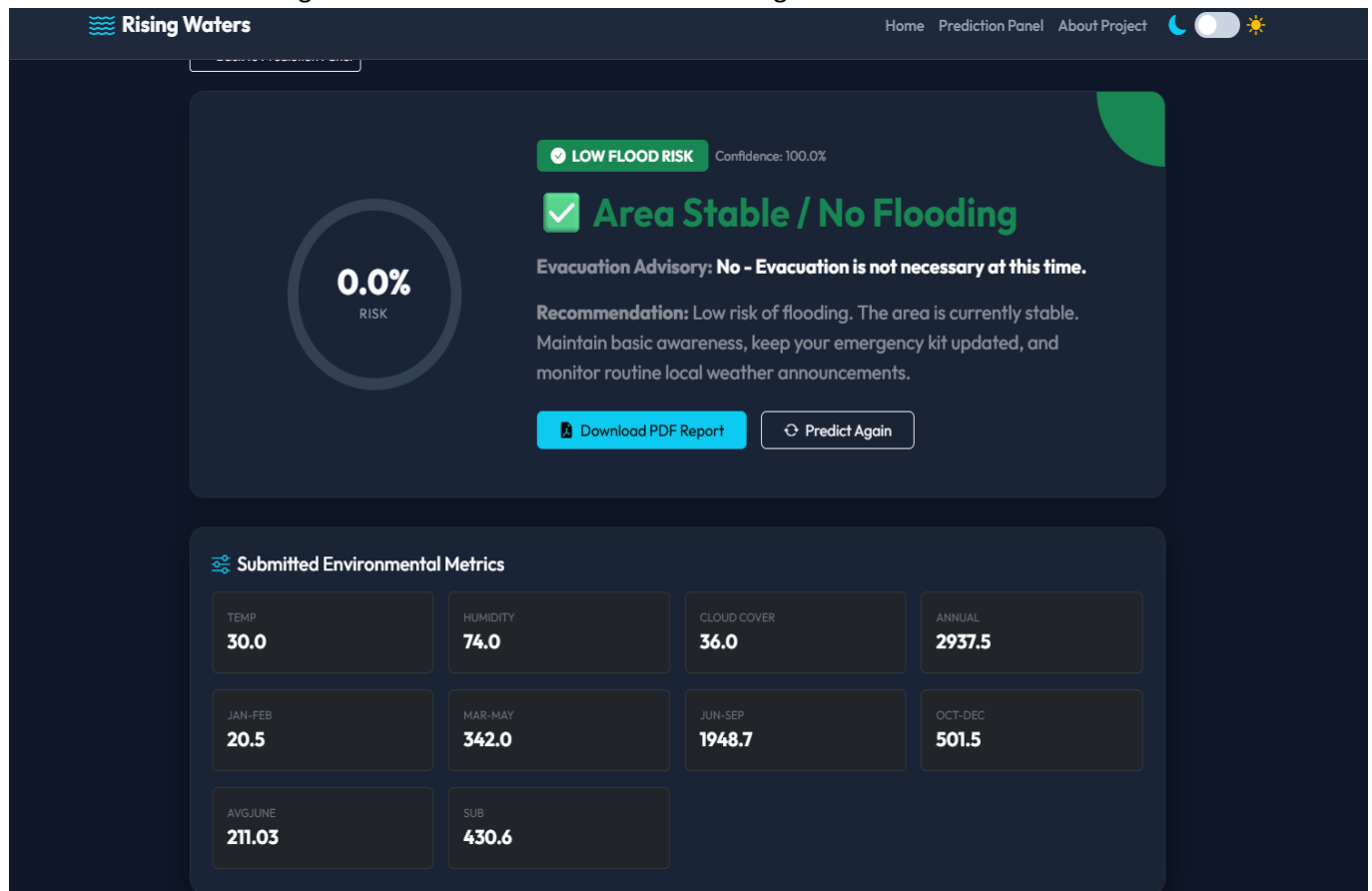
After the prediction is completed, the application displays the flood risk assessment generated by the trained Decision Tree model. The result page shows the Flood Risk Level, Risk Percentage, Prediction Confidence, Evacuation Advisory, and Safety Recommendations. It also displays the submitted weather parameters and provides options to download the prediction report as a PDF or perform another prediction.

Results Displayed

- Flood Risk Level (High Risk / Low Risk)
- Risk Percentage
- Prediction Confidence
- Evacuation Advisory

- Safety Recommendations
- Submitted Environmental Parameters
- Download PDF Report
- Predict Again Option

Figure 5.3: Flood Risk Prediction Result Page



Activity 5.4: Project Achievements

The Rising Waters project successfully achieved the objectives defined during the planning and development phases.

Key Achievements

- Developed a complete Machine Learning-based Flood Prediction System.
- Trained and evaluated Decision Tree, Random Forest, KNN, and XGBoost models.
- Achieved 95.65% prediction accuracy using the Decision Tree classifier.
- Developed a responsive Flask web application.
- Implemented client-side and server-side input validation.
- Integrated real-time flood risk prediction with safety recommendations.
- Enabled PDF report generation for prediction results.

Activity 5.5: Advantages of the System

The developed application offers several advantages.

- High prediction accuracy using Machine Learning.
- User-friendly and responsive web interface.
- Fast and real-time flood risk prediction.
- Provides prediction confidence and risk percentage.
- Displays evacuation advisories and safety recommendations.
- Generates downloadable PDF reports.
- Can support disaster preparedness and early warning systems.

Activity 5.6: Limitations

Although the application performs effectively, a few limitations remain.

- Uses a historical dataset for training.
- Does not integrate real-time weather data.
- No IoT sensor connectivity for live monitoring.
- No user authentication or login functionality.
- Currently deployed only in a local environment.
- Flood risk visualization on maps is not available.

Activity 5.7: Future Enhancements

The system can be further improved with the following enhancements.

- Cloud deployment for public accessibility.
- Integration with real-time weather APIs.
- IoT-based rainfall and river level monitoring.
- SMS and email alert notifications.
- GIS-based flood risk mapping.
- Android and iOS mobile applications.
- Historical flood analytics dashboard.
- AI-powered disaster management and resource planning.

Conclusion

Rising Waters: A Machine Learning Approach to Flood Prediction is an intelligent web application developed to predict flood risk using historical weather and rainfall data. The system integrates a Decision Tree Machine Learning model with a Flask-based web application to provide accurate and real-time flood risk predictions.

Multiple Machine Learning algorithms were trained and evaluated, with the Decision Tree model achieving an accuracy of 95.65%. The application successfully combines data preprocessing, model prediction, and result visualization while providing prediction confidence, flood risk percentage, evacuation advisories, and safety recommendations through an intuitive user interface.

The successful implementation of this project demonstrates the practical use of Machine Learning in disaster management. With future enhancements such as cloud deployment, real-time weather integration, IoT support, and GIS-based flood mapping, the system can evolve into a comprehensive flood early warning and decision-support solution.

Appendix

Appendix A: Software Requirements

Software	Version
Operating System	Windows 10/11
Python	3.12
Jupyter Notebook	Latest
Visual Studio Code	Latest
Flask	Latest
Pandas	Latest
NumPy	Latest
Scikit-learn	Latest
Matplotlib	Latest
Seaborn	Latest
XGBoost	Latest
Joblib	Latest
HTML5	Latest
CSS3	Latest
Bootstrap 5	Latest
JavaScript	ES6

Appendix B: Hardware Requirements

Hardware	Specification
Processor	Intel Core i5 or higher
RAM	Minimum 8 GB
Storage	Minimum 20 GB Free Space
Operating System	Windows 10/11 (64-bit)
Internet	Required for package installation

Appendix C: Machine Learning Algorithms Used

The project evaluates multiple Machine Learning algorithms for flood prediction.

- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

The **Decision Tree** model achieved the best overall performance with an accuracy of **95.65%** and was selected for deployment in the Flask application.

Appendix D: Project Folder Structure

Rising Waters/

|

|— app.py

|— train_model.py

|— requirements.txt

- |— README.md
- |
- |— dataset/
 - | flood_dataset.xlsx
- |
- |— model/
 - | flood_prediction.pkl
 - | scaler.pkl
 - | model_comparison.csv
- |
- |— notebooks/
 - | FloodPrediction.ipynb
- |
- |— static/
 - | css/
 - | js/
 - | images/
- |
- |— templates/
 - | base.html
 - | index.html
 - | predict.html
 - | result.html

```
| about.html
| error.html
|
└─ utils/
    preprocessing.py
```

GitHub Repository

<https://github.com/r210152-pixel/Rising-Waters-A-Machine-Learning-Approach-to-Flood-Prediction>

Appendix E: Model Performance

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Decision Tree	95.65%	1.00	0.67	0.80	0.8333
Random Forest	95.65%	1.00	0.67	0.80	0.9417
KNN	86.96%	0.50	0.33	0.40	0.7833
XGBoost	86.96%	0.00	0.00	0.00	0.5000

Appendix F: References

1. ☐ Python Documentation – <https://docs.python.org/3/>
2. ☐ Scikit-learn Documentation – <https://scikit-learn.org/>
3. ☐ Flask Documentation – <https://flask.palletsprojects.com/>
4. ☐ Pandas Documentation – <https://pandas.pydata.org/>
5. ☐ NumPy Documentation – <https://numpy.org/>
6. ☐ Matplotlib Documentation – <https://matplotlib.org/>
7. ☐ XGBoost Documentation – <https://xgboost.readthedocs.io/>
8. ☐ Bootstrap Documentation – <https://getbootstrap.com/>
9. ☐ Jupyter Notebook – <https://jupyter.org/>
10. SmartBridge AI & ML Learning Resources